

Solution Requirements

Date	
Team ID	xxxxxx
Project Name	EduGenie: Google Gemini Powered Learning Assistant
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

- Functional Requirements (FR) define specific behaviors, functions, or features of the system (What the system should do).
- Non-Functional Requirements (NFR) specify the criteria that judge the operation of a system (How the system should be).

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	No user login is required - EduGenie is an open learning tool. The server authenticates to Google Gemini with an API key kept in a private .env file (never in source code or the browser).	Medium
2	Authorization	Single role: Learner - can use all five features. The developer/administrator configures the API key, model and explanation backend through .env. No personal data is stored, so no role-based data access is needed.	Low
3	External interfaces	Google Gemini API via the google-genai SDK (Q&A, summary, quiz, learning path, explanation fallback); optional Hugging Face LaMini-Flan-T5-783M model (transformers + torch) for local explanations; browser-based HTML/CSS/JS frontend.	High
4	Transactions	Each request is stateless: input -> validation -> prompt construction -> AI call -> parsing/validation -> JSON response. Endpoints: GET /qa, POST /explain, POST /quiz, POST /summarize, GET /learn/recommendations, GET /health.	High
5	Reporting	Interactive quiz score shown after answering; GET /health reports configuration status; auto-generated API documentation at /docs; server logs for errors.	Medium
6	Business rules, etc.	Quiz: 1-10 MCQs (default 3), exactly 4 options, answer must match one option, invalid items discarded. Learning path levels: beginner / intermediate / advanced. Input limits: 300 characters for topics, 20,000 for passages. Empty input returns HTTP 400.	High
7	Compliance to laws or regulations	No personal data is collected or stored (privacy by design); API keys are kept out of version control (.gitignore); usage follows the Google Gemini API Terms of Service; AI output is presented as learning assistance.	Medium
8	Other	Copy result to clipboard, example inputs per task, error messages shown in the UI, keyboard shortcut (Ctrl/Cmd + Enter) to submit.	Low

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Pages and non-AI endpoints respond quickly; AI responses depend on Gemini latency. Blocking AI calls run in a thread pool so the server stays responsive.	Page load < 2 s; server overhead < 100 ms; AI answer typically < 10 s
2	Scalability	Stateless FastAPI app - can run multiple Uvicorn workers or containers behind a load balancer.	Handles 50 concurrent users with 0% errors (load test)
3	Security & Data Privacy	API key only on the server (.env, git-ignored); HTML output escaped in the browser to prevent XSS; input length limits; no user data stored.	0 secrets in repository; all AI text rendered escaped
4	Reliability & Availability	Graceful error handling: missing key -> 503, Gemini failure / invalid quiz -> 502 with a clear message; local model falls back to Gemini.	No unhandled exceptions; 31 automated tests pass
5	Usability & Accessibility	Simple single-page UI, responsive (mobile to desktop), dark mode, labels for all inputs, aria-live result area, keyboard support.	Usable at 360 px width; WCAG 2.1 AA colour contrast
6	Other (Maintainability, Portability)	Modular code (one module per feature), configuration via environment variables, runs on Windows / macOS / Linux with Python 3.10+.	Setup in < 10 minutes following README